

Dylan Shaffer

COSC 522

Project 1

AI Statement: AI assistance was used in this project. Autocomplete in VSCode was used, but only to quickly recreate functions that were similar (e.g., I wrote the function to predict type, autocomplete took that and changed the function name to work for hp, attack, and defense). Autocompleted functions have been commented as such in the code. Claude was used to assist with the predict function. Please note that I fully understand the math behind the code, I just needed help turning it into Python. Transcript from Claude chat posted below.

DylanShaffer_Project1.ipynb

260 lines

ipynb

I need help with this project. I need to predict a Pokemon's class based off of their HP, Attack, and Defense ratings. I have the probability of the type itself, and those attributes given the type.

I'm stuck trying to find out how to predict the type based on those attributes

10:07 AM

Looking at your code, you've already done the hard part — you have the prior probabilities and the likelihoods. Now you just need to apply **Bayes' theorem** to complete the predict function.

The idea is: for each type (Normal, Water, Flying), calculate:

$$P(\text{Type} \mid \text{HP, Attack, Defense}) \propto P(\text{Type}) \times P(\text{HP}|\text{Type}) \times P(\text{Attack}|\text{Type}) \times P(\text{Defense}|\text{Type})$$

You pick whichever type gives the highest score. Here's the completed predict function:

python

```
def predict(row):
```

```
    hp = row['HP']
```

```
    attack = row['Attack']
```

```
    defense = row['Defense']
```

```
    best_score = -1
```

```

best_class = "Unknown"

for tt in prior_probability_type.keys():
    # Start with the prior probability of this type
    score = prior_probability_type[tt]

    # Multiply by P(HP | Type) — use 0 if this HP value was never seen in training
    score *= probability_hp_gt[tt].get(hp, 0)

    # Multiply by P(Attack | Type)
    score *= probability_attack_gt[tt].get(attack, 0)

    # Multiply by P(Defense | Type)
    score *= probability_defense_gt[tt].get(defense, 0)

    if score > best_score:
        best_score = score
        best_class = tt

return {"score": best_score, "class": best_class}

```

A few things worth knowing:

The `.get(value, 0)` pattern handles unseen values gracefully — if a test Pokémon has an HP value that never appeared in training, it returns 0 instead of crashing. However, this means the whole score becomes 0 for that type, which can cause ties or misclassifications.

Zero-probability problem: If any attribute value is unseen for a type, the entire product becomes 0. A common fix is **Laplace (add-1) smoothing**, where instead of `count/total` you use `(count + 1) / (total + num_unique_values)`. This avoids zero probabilities for unseen values.

Why this works: You don't need to divide by $P(\text{HP, Attack, Defense})$ because it's the same for all types — you're just comparing scores, so the denominator cancels out. That's the "naïve" shortcut.